

AI VIDEO INTELLIGENCE PLATFORM

A Build-Ready Architecture, Evaluation Plan, Open-Source
Map,
and Coding-Agent Specification

**Focused MVP: Industrial safety and immediate risk prediction using existing
security cameras**

Prepared July 2026

Curated from the Virtual Lab repository, the uploaded PINNsAgent and PHIA papers, and current
open-source/project research.

Executive Summary

The recommended product is an edge-capable intelligence layer for existing IP cameras. It should detect people and vehicles, maintain short event histories, apply explicit safety rules, estimate near-term trajectory risk, generate evidence-backed incidents, and learn from operator feedback. The language model belongs above the analytics stack as a policy translator and explanation layer; it should not perform frame-by-frame detection or make unsupported claims about human intent.

Core product thesis

Build one coordinator and a reliable toolbox - not a society of agents. The operational truth must come from detectors, trackers, geometry, temporal models, calibrated risk functions, and human review.

What is borrowed - and what is deliberately left out

Source	Useful design pattern	Do not copy
Virtual Lab	Clear role separation during development: perception, event logic, evaluation, deployment, and domain review.	Multiple LLMs debating every camera event in production.
PINNsAgent	Versioned experiment memory, configuration reuse, feedback-driven optimization, and reproducible model evaluation.	PDE-specific encodings, PINN architecture search, or memory-tree search for the MVP.
PHIA	An LLM planner translates requirements and invokes trusted domain tools; outputs are explainable and visual.	A power-electronics surrogate or an LLM acting as the numerical engine.

Initial market and event scope

- Industrial warehouses and manufacturing sites using existing cameras.
- Restricted-zone entry.
- Loitering in a defined zone.
- Pedestrian-vehicle proximity.
- Predicted pedestrian-vehicle collision risk using calibrated trajectories.

Avoid overclaiming

Do not market the MVP as general crime prediction, intent recognition, emotion recognition, or autonomous emergency dispatch. The first product should estimate immediate, physically observable operational risk.

1. Product Definition

1.1 User outcome

A security or safety operator defines a policy in plain language or through a form. The platform compiles that policy into a constrained schema, validates cameras and zones, observes live video, evaluates the policy, and produces a reviewable incident with a short clip, annotated snapshot, structured evidence, confidence, and recommended operator action.

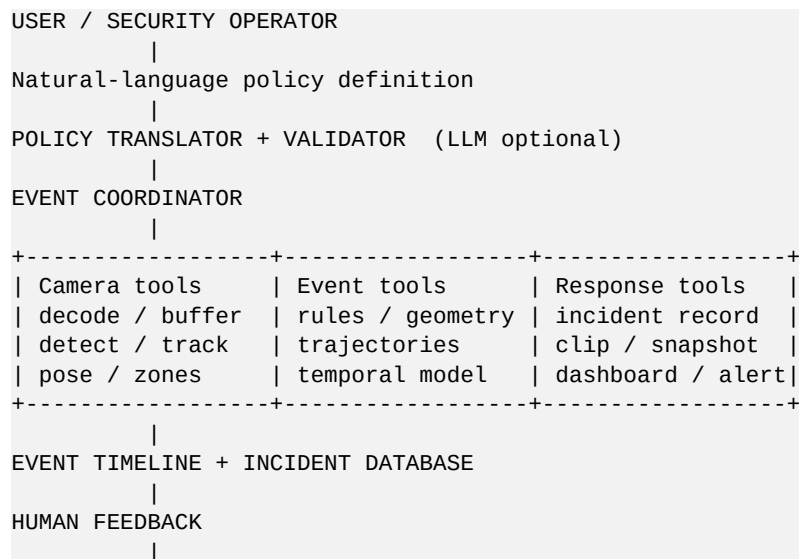
1.2 Three levels of intelligence

Level	Capability	Example
Observation	Detect visible entities and state changes.	A person entered Forklift Lane B.
Context	Reason over a short sequence and site policy.	The person remained for 8 seconds while a forklift began moving.
Near-term prediction	Project observable trajectories and estimate immediate risk.	Predicted closest separation is 0.8 m in 1.6 seconds.

1.3 Non-goals for the MVP

- Face recognition or biometric identity.
- Emotion, personality, or criminal-intent inference.
- Cross-city or cross-customer identity tracking.
- Automatic online training that changes production behavior without review.
- Direct emergency dispatch without a verified workflow.
- A general-purpose video foundation model trained from scratch.

2. Lean System Architecture



2.1 Production runtime rule

Real-time path

decode -> detect -> track -> update state -> evaluate rules -> emit event. The path must continue when the LLM, dashboard, or internet is unavailable.

2.2 LLM boundaries

Allowed	Not allowed
Translate natural-language policy into validated JSON/YAML.	Assign track IDs or calculate distances.
Ask for missing policy details.	Invent observations not present in tool outputs.
Explain why an incident fired.	Override thresholds or silently alter policies.
Summarize incident timelines and query structured records.	Continuously analyze every frame or directly trigger emergency action.

3. Recommended Repository Structure

```

sentinel-platform/
  apps/
    api/           FastAPI routes and schemas
    worker/        camera workers, watchdog, pipeline
    web/           operator dashboard
  sentinel/
    video/         RTSP, decode, sampling, ring buffer
    perception/    detector, tracker, pose, calibration
    events/        state machines and event rules
    policies/      schema, validator, compiler, translator
    incidents/     clips, snapshots, explanations
    evaluation/    accuracy, latency, calibration, benchmarks
    storage/       PostgreSQL and object-store adapters
    messaging/     internal event bus and schemas
    observability/ logs, metrics, traces
  configs/        cameras, policies, environment
  datasets/       manifests and annotations
  scripts/        import, benchmark, export, test-video tools
  tests/          unit, integration, end-to-end, performance
  docs/           architecture, privacy, deployment, threat model

```

3.1 Initial technology choices

Function	MVP choice	Scale-up option
Video decode and clips	FFmpeg/OpenCV	GStreamer / NVIDIA DeepStream
Detection	Replaceable detector adapter	TensorRT-optimized detector
Tracking	ByteTrack	BoT-SORT or DeepStream

		tracker after benchmarking
Geometry	NumPy + OpenCV + Shapely	GPU kernels only if profiling justifies it
API	FastAPI + Pydantic	Same
Persistent data	PostgreSQL	Partitioning or time-series extension
Temporary state	Redis	Redis Streams; Kafka only at larger scale
Artifacts	Local filesystem or MinIO	S3-compatible storage
Evaluation	CVAT + FiftyOne + MLflow	Same with automated data pipelines
Deployment	Docker Compose	Kubernetes only when operational scale requires it

4. Core Data Contracts

4.1 Frame, detection, and track

```

@dataclass(frozen=True)
class FramePacket:
    camera_id: str
    frame_id: int
    timestamp_utc: datetime
    image: np.ndarray
    width: int
    height: int
    source_fps: float

@dataclass(frozen=True)
class Detection:
    class_name: str
    confidence: float
    box_xyxy: tuple[float, float, float, float]

@dataclass
class TrackObservation:
    camera_id: str
    track_id: int
    timestamp_utc: datetime
    class_name: str
    confidence: float
    box_xyxy: tuple[float, float, float, float]
    ground_point_px: tuple[float, float]
    ground_point_m: tuple[float, float] | None
    velocity_mps: tuple[float, float] | None

```

For people and vehicles, use the bottom-center of the bounding box as the approximate ground-contact point. The box centroid is usually a poor choice for ground-plane geometry.

4.2 Event candidate and incident

```
@dataclass
class EventCandidate:
    event_type: str
    camera_id: str
    timestamp_utc: datetime
    involved_track_ids: list[int]
    confidence: float
    severity: str
    evidence: dict[str, Any]
    policy_id: str

{
  "event_type": "collision_risk",
  "camera_id": "warehouse_cam_03",
  "confidence": 0.86,
  "track_ids": [127, 203],
  "observations": {
    "current_distance_m": 2.4,
    "predicted_minimum_distance_m": 0.72,
    "time_to_minimum_distance_s": 1.6,
    "relative_speed_mps": 2.8
  },
  "triggered_conditions": [
    "predicted_minimum_distance_m < 1.0",
    "time_to_minimum_distance_s < 2.0"
  ],
  "review_status": "unreviewed"
}
```

5. Policy Language and Coordinator

5.1 Natural-language example

"Alert me when a person enters the forklift lane, remains there for more than five seconds, and a moving forklift is predicted to come within one meter during the next three seconds."

5.2 Compiled policy

```
policy_id: forklift_person_risk_v1
scope:
  camera_ids: [warehouse_cam_03]
subjects:
  primary: {class: person}
  secondary: {class: forklift}
zones:
  include: [forklift_lane]
conditions:
  all:
    - {type: dwell_time, subject: primary, minimum_seconds: 5}
    - {type: object_motion, subject: secondary, minimum_speed_mps: 0.3}
    - type: predicted_separation
      subjects: [primary, secondary]
      horizon_seconds: 3
      threshold_meters: 1.0
event:
  type: collision_risk
  cooldown_seconds: 30
  severity: high
```

```
evidence:
  pre_event_seconds: 8
  post_event_seconds: 8
  save_clip: true
  save_snapshot: true
review:
  human_confirmation_required: true
```

5.3 Validation rules

- Reject unknown camera IDs, zones, object classes, or event types.
- Reject negative durations and impossible schedules.
- Reject metric-distance policies for uncalibrated cameras.
- Reject LLM output that fails the strict Pydantic/JSON schema.
- Never execute generated Python.
- Return accepted, rejected, or accepted-with-warnings plus a human-readable explanation.

6. Real-Time Perception Pipeline

```
async for frame in source.frames():
    if not frame_sampler.accept(frame):
        continue
    detections = detector.predict(frame.image)
    tracks = tracker.update(detections, frame)
    observations = observation_builder.build(tracks, calibration)
    candidates = event_engine.update(observations)
    for candidate in candidates:
        await incident_queue.publish(candidate)
```

6.1 Processing rates

Stage	Initial rate	Reason
Decode	15 FPS	Smooth tracking and clip playback.
Detector	5 FPS	Reduces GPU cost; tracker fills intermediate frames.
Tracker	15 FPS	Maintains trajectories between detections.
Event engine	15 FPS	Keeps state and risk timing responsive.

6.2 Camera health

- Last frame received and last successful inference.
- Decode FPS, inference FPS, and dropped-frame rate.
- Reconnect attempts and time since disconnect.
- Worker heartbeat, queue depth, disk capacity, and GPU-memory status.

7. Event Engine

7.1 Restricted-zone entry

1. Calculate the ground-contact point.
2. Test whether it lies inside the polygon.
3. Maintain previous and current inside/outside state.
4. Require a stable transition and debounce boundary jitter.
5. Apply minimum duration, cooldown, and deduplication.

```
OUTSIDE -> PENDING_ENTRY -> INSIDE -> PENDING_EXIT -> OUTSIDE
```

7.2 Loitering

Track stable zone-entry time and accumulate dwell duration. Support a small tracking-gap tolerance so brief occlusion does not reset the timer. The policy must define whether a true exit resets, pauses, or closes the incident.

7.3 Proximity

When metric calibration exists, calculate Euclidean separation between ground-plane positions. Require sufficient track history, acceptable tracking confidence, and meaningful vehicle motion before alerting.

7.4 Collision-risk prediction

Begin with constant-velocity projection. This is transparent, inexpensive, and easier to validate than an opaque video model.

```
r = person_position - vehicle_position
u = person_velocity - vehicle_velocity

t_star = clip(-(r dot u) / (||u||^2 + epsilon), 0, horizon)
d_min = ||r + t_star * u||

trigger when:
    d_min < distance_threshold
    and t_star < time_threshold
    and vehicle_speed > minimum_speed
```

Why this is real prediction

The system predicts short-horizon physical convergence from measured trajectories. It does not claim to predict a person's criminal intent.

7.5 Later learned risk score

After enough labeled site data exists, a calibrated logistic-regression or gradient-boosting model can combine predicted distance, time to closest approach, relative speed, occlusion, zone type, and historical context. Keep the original geometric features in every incident for explainability.

8. Camera Calibration

A monocular camera does not automatically provide reliable meters. For planar floors, use a homography derived from at least four image points and their real-world coordinates. Store the matrix, resolution, calibration image, operator, date, and reprojection error.

```
image_points = [[131,702],[997,705],[844,355],[302,359]]
world_points_m = [[0,0],[10,0],[10,15],[0,15]]
```

- Reject metric-risk policies when calibration is missing or reprojection error exceeds the threshold.
- Recalibrate after a camera is moved, zoomed, or its resolution changes.
- Expose uncertainty when the floor is non-planar or objects move on ramps.

9. Incident Evidence and Storage

9.1 Evidence bundle

- Event type, camera, zone, start/end time, and severity.
- Involved anonymous track IDs and structured measurements.
- Triggered conditions and threshold values.
- Pre-event and post-event clip.
- Annotated snapshot with zones, tracks, and projected trajectories.
- Model, policy, calibration, and code versions.
- Human-review status and feedback.

9.2 Ring-buffer flow

```
past 10-20 seconds + live post-event frames -> incident clip
```

9.3 Minimum database tables

```
users, sites, cameras, camera_calibrations, zones, policies, policy_versions, tracks,
track_observations, event_candidates, incidents, incident_entities, incident_evidence,
operator_feedback, model_versions, evaluation_runs, camera_health
```

Storage rule

Do not store every frame in PostgreSQL. Store metadata and sampled observations there; store clips, images, datasets, and reports in object storage.

10. API Surface

POST /cameras	GET /cameras/{id}/health
POST /cameras/{id}/calibrations	POST /zones
POST /policies/translate	POST /policies/{id}/validate
POST /policies/{id}/activate	GET /incidents
GET /incidents/{id}	GET /incidents/{id}/clip
POST /incidents/{id}/feedback	POST /evaluation/runs
GET /evaluation/runs/{id}/report	GET /metrics

11. Human Feedback Loop

```
{
```

```

"incident_id": "uuid",
"verdict": "true_positive",
"correct_event_type": "collision_risk",
"severity_assessment": "medium",
"usefulness": 4,
"comments": "Valid near miss, but severity was too high."
}

```

- Allowed verdicts: true positive, false positive, wrong severity, wrong event type, insufficient evidence, camera-quality failure, and duplicate incident.
- Feedback enters an offline evaluation dataset. It must not directly modify production thresholds.
- Every production change requires a versioned evaluation run and rollback path.

12. Evaluation Framework

12.1 Four independent scorecards

Scorecard	Core measures
Perception	Precision, recall, AP, miss rate by object size, lighting, and occlusion.
Tracking	HOTA, IDF1, identity switches, fragmentation, and lost tracks per camera-hour.
Events	Event precision/recall/F1, temporal IoU, false alerts per camera-day, delay, duplicates.
Operations	P50/P95/P99 latency, FPS, dropped frames, GPU/CPU/RAM, reconnect time, uptime.

12.2 Event matching

Match a predicted incident to ground truth when camera and event type match and temporal intersection-over-union exceeds a declared threshold. For trajectory-risk events, also evaluate minimum-distance error and time-to-closest-approach error.

12.3 Suggested MVP targets

Measure	Engineering target
Restricted-zone precision and recall	$\geq 95\%$ on held-out staged data
Loitering recall	$\geq 90\%$
Collision-risk recall	$\geq 85\%$ initially
False alerts	≤ 1 per camera-day per policy
P95 incident latency	≤ 2 seconds
Camera reconnect	≤ 30 seconds
Dropped frames	$\leq 2\%$
Evidence clip availability	$\geq 99\%$

24-hour worker stability	No sustained memory or GPU-memory leak
--------------------------	--

12.4 Runtime metrics

decode_fps, inference_fps, tracking_fps, end_to_end_fps, p50/p95/p99_latency, gpu_utilization, gpu_memory, cpu, ram, network, dropped_frames, reconnect_time

12.5 Reproducibility contract

Definition of working

The same versioned dataset + configuration + model + code commit must produce reproducible event and runtime metrics. Boxes on a demo video are not sufficient evidence that the platform works.

13. Dataset and Test Design

13.1 Required slices

- Day, low light, glare, motion blur, camera vibration, and partial occlusion.
- Crowded and uncrowded scenes.
- Boundary crossings lasting one frame versus genuine entry.
- Stationary vehicle, moving-away vehicle, and converging trajectories.
- Tracking interruption, object partially outside frame, and reflections.

13.2 Collision-risk hard negatives

- Parked forklift beside a worker.
- Forklift moving away.
- Person and forklift crossing the same point at different times.
- Worker behind a physical safety barrier.
- Loading activity near a stationary vehicle.
- Perspective that visually compresses distance.

13.3 Automated tests

Test type	Examples
Unit	Point-in-polygon, dwell timer, velocity, closest approach, cooldown, deduplication.
Integration	Video -> detector -> tracker -> event -> incident DB and clip.
End-to-end	Synthetic RTSP stream -> worker -> API -> dashboard.
Soak	24-hour run for memory, handles, queue growth, and GPU stability.
Failure injection	RTSP loss, DB outage, full disk, detector exception,

	unavailable LLM.
--	------------------

14. Development Roadmap and Acceptance Gates

Phase	Deliverable
Phase 0 - Foundation	Repository, Docker Compose, PostgreSQL, Redis, storage, health endpoint, typed config, structured logs, CI.
Phase 1 - Recorded-video perception	MP4 input, detector adapter, ByteTrack adapter, annotated output, JSONL tracks, latency/FPS report.
Phase 2 - Zones and rules	Zone editor/config, restricted entry, loitering, debounce, cooldown, unit tests.
Phase 3 - Incidents	Ring buffer, pre/post clip, snapshot, incident DB, API, simple incident list.
Phase 4 - Calibration and risk	Homography, metric positions, smoothing, velocity, proximity, closest-approach risk.
Phase 5 - Evaluation	Dataset manifest, event matching, reports, failure lists, MLflow logging.
Phase 6 - Policy translator	Structured LLM output, validation, preview, confirmation, graceful no-LLM mode.
Phase 7 - Live pilot	RTSP reconnection, health dashboard, shadow mode, 24-hour soak, operator feedback.

14.1 Build order rule

Do not skip forward

Do not add action recognition or a vision-language verifier until deterministic events, incident evidence, and evaluation are reliable. Do not add DeepStream until Python correctness and the throughput bottleneck are measured.

15. Open-Source Repository Map

The repositories below are intended as references or replaceable components. Do not combine all of them into one monolith. Review each license and current maintenance status before commercial use.

Repository	Purpose	Recommended use	URL
zou-group/virtual-lab	Scientific team/meeting pattern	Use role separation for development and review; not live multi-	https://github.com/zou-group/virtual-lab

		agent inference.	
blakeblackshear/frigate	Local NVR, RTSP, zones, recording, efficient inference	Study stream ingestion, motion gating, buffering, review, and camera recovery.	https://github.com/blakeblackshear/frigate
FoundationVision/ByteTrack	Multi-object tracking	Use or adapt the tracker; benchmark ID switches and occlusion behavior.	https://github.com/FoundationVision/ByteTrack
ultralytics/ultralytics	Rapid detection, pose, tracking integrations	Useful for prototypes; perform a careful commercial license review.	https://github.com/ultralytics/ultralytics
open-mmlab/mmdetection	Action recognition and temporal localization	Add only for events that rules and trajectories cannot represent.	https://github.com/open-mmlab/mmdetection
open-mmlab/mmdetection	Tracking research framework	Reference for experiments; likely too heavy for MVP runtime.	https://github.com/open-mmlab/mmdetection
mikel-brostrom/boxmot	Tracker comparison	Use to benchmark tracker families, then select one for production.	https://github.com/mikel-brostrom/boxmot
cvat-ai/cvat	Video annotation and review	Primary annotation tool for boxes, tracks, zones, and QA.	https://github.com/cvat-ai/cvat
voxel51/fiftyone	Dataset and failure analysis	Inspect predictions, filter hard examples, compare model versions.	https://github.com/voxel51/fiftyone
mlflow/mlflow	Experiment and model registry	Record code, data, model, thresholds, metrics, and artifacts.	https://github.com/mlflow/mlflow
NVIDIA/DeepStream	High-density GPU video analytics	Introduce when multi-stream scale justifies GStreamer/TensorRT complexity.	https://github.com/NVIDIA-AI-IOT/deepstream_python_apps
facebookresearch/pytorchvideo	Video-model research	Alternative reference for PyTorch-native video understanding.	https://github.com/facebookresearch/pytorchvideo

QwenLM/Qwen2.5-VL	Optional vision-language verification	Use only for selected ambiguous clips and descriptions, not core geometry.	https://github.com/QwenLM/Qwen2.5-VL
-------------------	---------------------------------------	--	---

15.1 Minimal MVP selection

Need	Choice
Decode/clips	FFmpeg and OpenCV
Detection	Replaceable detector adapter
Tracking	ByteTrack
Geometry	OpenCV + NumPy + Shapely
API/config	FastAPI + Pydantic
Data	PostgreSQL + Redis + MinIO/local
Annotation	CVAT
Inspection	FiftyOne
Experiments	MLflow
Monitoring	Prometheus + Grafana
Later scale	DeepStream

16. Company Landscape and Positioning

The commercial market is active. A new company should not differentiate merely by saying it adds AI to cameras. The practical opening is a narrow, measurable risk workflow with retrofit deployment, transparent evidence, and site-specific performance evaluation.

Company/group	What they do	Strategic lesson
Verkada	Integrated cameras, cloud management, access control, search, analytics.	Strong end-to-end experience; typically centered on its ecosystem.
Rhombus	Cloud-managed cameras, sensors, access control, AI search and alerts.	Integrated physical-security platform.
Avigilon / Motorola Solutions	Enterprise cameras, VMS, analytics, access control, public-safety ecosystem.	Large established enterprise suite.
Ambient.ai	AI threat-detection layer over existing cameras and access-control systems.	Close to retrofit contextual security intelligence.

Spot AI	Video intelligence for safety, operations, investigations, and existing camera fleets.	Operational video analytics beyond traditional security.
BriefCam	Forensic search, video synopsis, analytics, and investigation acceleration.	Strong retrospective search and review.
ZeroEyes	Visible-firearm detection with trained human verification and alert workflow.	Demonstrates value of a narrow high-consequence event and human-in-loop response.
Genetec / Milestone ecosystems	Video-management platforms supporting third-party integrations.	Potential distribution/integration channel rather than direct replacement.

16.1 Recommended differentiation

Positioning

An edge-deployable industrial safety layer for existing cameras that predicts immediate trajectory-based risk, explains the physical sequence behind every alert, and continuously measures site-specific false-alert and miss rates.

16.2 Defensible product capabilities

- Physics-grounded time-to-collision and predicted-separation evidence.
- Site calibration and uncertainty reporting.
- Customer-defined policies that compile to transparent event logic.
- Shadow-mode validation before alerts are activated.
- Per-camera and per-policy performance reporting.
- Open VMS/RTSP integration and local deployment.
- Hard-negative learning from operator feedback without unsafe online updates.

17. Security, Privacy, and Responsible Use

- Default to anonymous track IDs and avoid biometric identity.
- Encrypt camera credentials, clips, and data in transit and at rest.
- Use least-privilege service accounts and separate tenants/sites.
- Provide retention controls, audit logs, and deletion workflows.
- Restrict policy creation, activation, and alert destinations by role.
- Treat VLM-generated descriptions as unverified unless grounded in structured evidence.
- Document camera blind spots, calibration uncertainty, and known failure modes.
- Require human verification for high-consequence actions.

17.1 Threat model checklist

Threat	Control
--------	---------

Stolen RTSP credentials	Secrets manager, rotation, no plaintext logs.
Malicious policy prompt	Constrained schema, allowlists, no code execution.
Adversarial or misleading footage	Confidence thresholds, camera-health checks, human review.
Model/config regression	Versioning, holdout tests, canary/shadow deployment, rollback.
Data leakage	Tenant isolation, access logs, retention and export controls.
Alert flooding	Cooldown, deduplication, rate limits, policy-level false-alert monitoring.

18. Coding-Agent Master Specification

The following prompt can be provided to a coding agent as the top-level project instruction.

You are the lead software engineer for an edge-capable security-camera video intelligence platform.

Build incrementally. Do not create an overengineered multi-agent runtime. The production path is deterministic: video ingestion, detection, tracking, event state, risk calculation, incident evidence, evaluation, and optional natural-language policy translation.

INITIAL EVENTS

1. restricted-zone entry
2. loitering
3. pedestrian-vehicle proximity
4. predicted pedestrian-vehicle collision risk

NON-GOALS

No face recognition, emotion inference, general crime prediction, weapon/fight detection, cross-camera identity, automatic dispatch, or online production retraining.

TECHNICAL REQUIREMENTS

- Python 3.12, strong typing, FastAPI, Pydantic.
- PostgreSQL + SQLAlchemy + Alembic.
- Redis for temporary state and queues.
- FFmpeg/OpenCV initially; model-specific code behind interfaces.
- ByteTrack-compatible tracker.
- S3-compatible storage interface.
- Docker Compose, Prometheus metrics, structured JSON logs.
- Unit, integration, end-to-end, performance, soak, and failure tests.
- No arbitrary execution of LLM-generated code.
- No LLM dependency in frame processing.
- UTC timestamps and versioned policies/models/configurations.

IMPLEMENTATION PROCESS

Before each phase: state objective, files, acceptance tests, assumptions.
After each phase: run formatting, type checking, unit and integration tests; report exact commands and honest results. Never claim unrun tests passed.

EVENT RULES

- Restricted zone: bottom-center ground point, stable transition, debounce.
- Loitering: dwell state with configurable tracking-gap tolerance.
- Proximity: metric calibration, track history, motion threshold.
- Collision risk: smooth positions, estimate velocity, calculate time to closest approach and minimum predicted separation, store evidence.
- Reject metric policies on uncalibrated cameras.

POLICY TRANSLATION

Generate only schema-validated JSON/YAML selected from supported cameras, zones, subjects, conditions, and actions. Never generate executable code.

EVALUATION

Implement detection, tracking, event precision/recall/F1, temporal IoU, false alerts per camera-hour/day, delay, duplicate rate, trajectory error, confidence calibration, FPS/latency/resource use, dropped frames, and reconnect time. Record Git commit, model version, dataset version, thresholds, dependency lock, and camera configuration.

FIRST TASK

Create architecture.md, event_definition.md, evaluation_protocol.md, repository skeleton, pyproject.toml, docker-compose.yml, typed config, health endpoint, test infrastructure, and a Phase 1 plan.

Do not implement action recognition or an LLM coordinator yet.

19. Decision Checklist Before Coding

Decision	Recommended default
First customer environment	One warehouse or manufacturing pilot.
Camera count	2-5 cameras in shadow mode.
Primary events	Restricted entry, loitering, proximity, collision risk.
Inference location	Local GPU edge server.
Identity	Anonymous per-camera track IDs.
Distance	Calibrated ground-plane homography.
Alerting	Dashboard first; no automatic emergency action.
VLM	Not in v1 real-time path.
Action recognition	Defer until rule/trajectory baseline is measured.
Success gate	Held-out metrics plus 24-hour RTSP stability.

20. Sources and Further Reading

Primary materials used for the conceptual synthesis:

- Virtual Lab repository: <https://github.com/zou-group/virtual-lab>
- Qingpo Wu et al., "PINNsAgent: Automated PDE Surrogation with Large Language Models," ICML 2025 (uploaded source).

- Junhua Liu et al., "Physics-Informed Autonomous LLM Agents for Explainable Power Electronics Modulation Design," AAAI 2026 (uploaded source).

Open-source projects and commercial sites should be rechecked immediately before adoption because versions, licenses, APIs, and product capabilities change. Repository and market information in this document was curated in July 2026.

Selected official links

- <https://github.com/blakeblackshear/frigate>
- <https://github.com/FoundationVision/ByteTrack>
- <https://github.com/open-mmlab/mmdetection>
- <https://github.com/cvat-ai/cvat>
- <https://github.com/voxel51/fiftyone>
- <https://github.com/mlflow/mlflow>
- <https://developer.nvidia.com/deepstream-sdk>
- <https://www.verkada.com>
- <https://www.rhombus.com>
- <https://www.avigilon.com>
- <https://www.ambient.ai>
- <https://www.spot.ai>
- <https://www.briefcam.com>
- <https://zeroeyes.com>